

COS 214 Practical 6

TIVIDY: Workflow Management System

Date Released:	29 September 2026
Date Due:	6 October 2026
Final Demo:	2 November 2026
Team Size:	5 or 7 students
Language:	C++11
Core topics:	Workflow research, software design, GoF design patterns, UML Activity Diagrams, UML Class Diagrams, Sequence Diagrams, State Diagrams, Git/GitHub

1 Introduction

Practical 6 marks the beginning of the **COS 214 project**. For the remainder of the semester, your group will design and develop a workflow management system called **TIVIDY**.

The final project requires you to combine the object-oriented design principles, UML modelling techniques and Gang of Four design patterns covered throughout COS 214 into one substantial software system. Unlike earlier practicals, you are not being given a predefined class structure. You are being given a problem that must first be understood and designed.

A workflow is a structured sequence of activities that transforms an input into an outcome. It describes what work must be performed, who or what performs it, in what order it occurs, under what conditions it may continue, and what happens when work succeeds, fails, is rejected or requires escalation.

A workflow management system stores workflow definitions and creates and manages running workflow instances from those definitions. Depending on the process, it may need to coordinate work items, participants, routing decisions, approvals, events, dependencies, external systems, history, monitoring and reporting.

Practical 6 is the research and initial design phase of the project. No implementation of the complete system is required for this practical.

2 Why Workflow Management?

Workflow management systems are used throughout modern organisations because most real work is not a collection of independent tasks. Work normally moves through a process involving multiple people, systems, decisions, approvals and dependencies.

Examples include insurance claims, university applications, employee onboarding, procurement, laboratory testing, manufacturing, logistics, customer support, maintenance, software delivery, document approval and regulatory processes.

Without a structured workflow, organisations often coordinate these processes through combinations of email, spreadsheets, messages and individual knowledge. This becomes difficult to control as the number of people, systems and process variations increases.

A workflow management system can help an organisation define how a process should operate, route work to the correct participants, enforce rules and dependencies, track progress, integrate external systems, maintain an audit history and identify delays or bottlenecks.

For this reason, workflow management is also a useful software-design problem. The system contains many responsibilities that must be able to vary independently, making it suitable for applying design patterns in a larger coherent architecture.

3 Project Structure

The project must be completed in groups of **5 or 7 students**. Each group must elect a group leader who will facilitate communication between the group, tutors and ALs. All members remain responsible for understanding the complete system.

Weekly project check-ins will be conducted to provide feedback, clarify requirements and review progress. **Additional requirements may be introduced during the project.**

4 TIVIDY

TIVIDY must be designed as a **generic workflow management system**. Your team will select one realistic workflow scenario through which the system can eventually be demonstrated. The scenario should use the workflow manager; it should not define the entire architecture of the workflow manager itself.

A useful starting point is to separate the **description of a process** from a **running process**. A workflow definition describes how a process is allowed to behave. The same definition may be used to create many independent workflow instances, each with its own participants, data, progress and history. A change to one running instance should therefore not accidentally change every other instance created from the same definition.

Work inside a workflow is also rarely flat. A process may contain stages, individual work items and smaller sub-processes. Some work may happen sequentially, some may happen in parallel, and some may only become available when earlier work has completed. Your design should therefore consider how individual pieces of work and groups of work are represented, owned and navigated without requiring the rest of the system to know their internal storage structure.

Different organisations may make different decisions about the same kind of work. One organisation may assign the next task based on role, another on workload, and another on a rule calculated from workflow data. Approval may involve one person, several levels, or escalation when nobody responds. These rules should not require the fundamental work-item classes to be rewritten each time the process changes.

Running work also changes over time. A work item may be created, made available, assigned, started, completed, rejected, cancelled or escalated. Operations that are valid at one point may be invalid at another. At the same time, a task may acquire additional behaviour such as validation, priority handling, auditing, notifications or security without becoming a completely different kind of task.

A workflow system must also react to what happens around it. Completing one activity may make another available, an external event may trigger new work, and several interested parts of the system may need to react to the same event. TIVIDY may also need to communicate with external systems whose interfaces were not designed for it. These concerns should be separated from the core workflow model so that new rules, interested parties or integrations can be introduced without redesigning the entire engine.

Finally, a real workflow system must be able to explain what happened. It may need to retain actions and state changes, support undo or rollback where sensible, preserve the meaning of older workflow definitions, traverse running structures for different views, and produce monitoring or reporting information without placing every possible reporting operation inside the core model.

These are examples of the **design forces** acting on TIVIDY. Your research may reveal others. The purpose of Practical 6 is to decide which abstractions are necessary, which responsibilities belong together, which should remain separate, and where the system must be able to change without widespread modification.

Possible concepts that may appear in a workflow system include:

- workflow definitions and running workflow instances;
- work items, stages and sub-processes;
- participants, roles and responsibilities;
- work-item lifecycles;
- routing and assignment rules;
- approvals, rejection, escalation and deadlines;
- events and triggers;
- data payloads, artefacts, constraints and dependencies;
- workflow construction and execution;
- actions, history and rollback;
- integrations and connectors;
- workflow versioning and traversal;
- access control and resource coordination; and
- monitoring, analytics and reporting.

This is **not a required class list**. Part of the project is deciding what belongs in the system and how those

responsibilities should collaborate.

5 Task 1: Research Workflow Management Systems

Before designing TIVIDY, your group must first understand the problem domain. Conduct further research into workflows and workflow management systems and submit concise research findings of approximately **one page**, excluding references.

Use at least **four credible references**. Your research should investigate questions such as:

- What is a workflow and what is a workflow management system?
- What is the difference between a workflow definition and a running workflow instance?
- How are activities or work items represented?
- How are work items assigned and routed?
- How are approvals, rejection and escalation handled?
- How are participants, roles and responsibilities represented?
- How are dependencies, events and failures handled?
- How do workflow systems interact with external systems?
- How is workflow history or audit information recorded?
- How are running workflows monitored and reported on?
- What makes a workflow engine reusable across different processes?
- Which information belongs to a workflow definition and which belongs to a running instance?
- How do real workflow systems model nested activities or sub-processes?
- Which decisions are normally configurable rather than hard-coded into the workflow engine?
- How are changes to workflow definitions handled while work is already in progress?

Do not simply collect definitions. Your research should help you understand **what must be designed**. Look specifically for responsibilities that repeat across different workflow products and for parts of the system that are expected to change independently. Where an important design idea originates from your research, reference it appropriately in the design documentation.

6 Task 2: Define the Scenario and System

Using your research, first define the initial scope of TIVIDY as a generic workflow management system. Identify the main responsibilities of the workflow manager, its boundaries, the information that belongs to a workflow definition versus a running instance, and the important parts of the system that you expect to vary.

Choose one realistic workflow scenario that will eventually be executed using TIVIDY. The scenario should be complex enough to exercise the generic system rather than becoming the system itself.

For the chosen workflow scenario, describe:

- the organisation or environment;
- the process being managed;
- the participants and roles involved;
- the major pieces of work;
- important decisions and routing points;
- approvals and dependencies;
- possible failures or exceptions;
- external systems where relevant; and
- the expected outcome of the workflow.

Begin with the workflow problem, not with pattern names. Your scenario is one application of TIVIDY, not the design of TIVIDY itself.

7 Task 3: UML Activity Diagrams

Before designing classes, model how the system behaves.

Create **three substantial UML Activity Diagrams**:

1. A generic workflow execution showing the progression of a workflow instance from creation or start until completion or another appropriate final outcome.
2. Another important generic TIVIDY workflow, such as work assignment and completion, approval and escalation, failure and recovery, or event handling.
3. An important workflow from your selected scenario.

Across the diagrams, demonstrate appropriate use of actions, decisions, guards, merges and loops, as well as at least one fork/join, one called or composite activity, and one use of swimlanes.

The diagrams should expose meaningful behaviour. Do not hide an entire process behind a single action such as `processWorkflow()`.

The diagrams are part of the design. Important actions and decisions shown in the diagrams must be traceable to the proposed class design.

8 Task 4: Design Patterns

The final TIVIDY system must make meaningful use of at least **10 Gang of Four design patterns**. Patterns that may be relevant include, but are not limited to:

- Abstract Factory, Adapter, Builder and Chain of Responsibility;
- Command, Composite, Decorator and Facade;
- Iterator, Mediator, Memento and Observer;
- Proxy, State, Strategy, Template Method and Visitor.

You are not required to use every pattern in this list. A pattern should appear because the forces in your design justify it.

For every proposed pattern, document:

- the design problem in TIVIDY that the pattern solves;
- the classes that will be incorporated in your design that fulfil the GoF participant roles;
- how those participants collaborate;
- what part of the design is expected to vary; and
- why the selected pattern is appropriate.

A statement such as “we use Strategy because Strategy allows interchangeable algorithms” is not sufficient. The justification must refer to the actual workflow-management problem being solved.

9 Task 5: UML Class Diagram

Use different colours or another clear visual notation in the UML Class Diagram to indicate important pattern collaborations or subsystems. Include a legend. A class may participate in more than one collaboration.

Create a complete UML Class Diagram for your proposed TIVIDY design. The diagram must be drawn from scratch and should show, where appropriate:

- classes and abstract classes;
- inheritance, associations, aggregation and composition;

- multiplicities;
- important attributes and operations;
- ownership relationships; and
- collaborations between the major parts of the system.

Do not design toward a fixed number of classes. The design should be as small as it can reasonably be while still supporting the required behaviours.

10 Task 6: Runtime Behaviour Diagrams

A class diagram shows what exists, but it does not fully explain how objects collaborate at runtime.

Create at least **two UML Sequence Diagrams** showing important interactions in the proposed system. Suitable interactions include:

- creating and starting a workflow instance;
- completing a work item and determining what happens next;
- executing an approval or escalation process;
- responding to an event;
- invoking an external integration; or
- another important interaction justified by your design.

The interactions should involve multiple parts of the design rather than demonstrating one class in isolation.

Create at least **one UML State Diagram** showing the lifecycle of an important runtime object. A suitable object may be a work item, workflow instance, approval request or another stateful object justified by your design.

The State Diagram should show meaningful states, transitions, triggers, guards where appropriate, and appropriate final states. Do not create states solely to demonstrate the State pattern.

11 Task 7: Design Decisions and Revisions

Your first design is not expected to be your final design. As your research, discussions and tutor feedback progress, you may discover incorrect assumptions or better solutions.

Document important design decisions and meaningful changes. For each major revision, briefly record:

- the problem or question being considered;
- the original design or assumption;
- the change that was made; and
- why the change was made; and
- updates to the relevant diagrams.

Your design history should continue throughout the project.

12 Task 8: GitHub Workflow

Create the GitHub repository for the project during Practical 6. The repository is both a development workspace and evidence of the team's engineering process.

At minimum, the repository should contain:

- `README.md`;
- a `docs/` directory;
- research material and references;
- UML diagrams or exported versions of them; and
- design decisions and revision history.

Do not create or populate the repository only at the end. Use meaningful commits and ensure that all group members make meaningful contributions.

13 Non-negotiable Rules

1. Complete the project in a team of **5 or 7**. Every member must understand the complete design.
2. TIVIDY must be designed as a **generic workflow management system**.
3. Your selected scenario must not determine the complete architecture of the generic workflow manager.
4. The final project must contain at least **10 meaningful GoF design patterns**.
5. Patterns must solve genuine design problems and must form part of one coherent system rather than isolated demonstrations.
6. Large centralised **if/else** chains, giant **switch** statements and manual type checks may not replace appropriate polymorphic design.
7. Your UML diagrams must agree with one another.
8. Important ownership and lifetime relationships must be clear in the design.
9. Important runtime behaviour must be visible and explainable from the diagrams.
10. Important design changes must be documented.
11. GitHub is compulsory and must show genuine work over time from the group.
12. No complete implementation is required for Practical 6.

14 Submission

Submit **one PDF**. The GitHub repository must remain accessible to the teaching team.

The PDF should contain, in a sensible order, at least:

- team details and GitHub repository link;
- workflow-management research findings and references;
- selected scenario description;
- initial TIVIDY system scope and design rationale;
- the three UML Activity Diagrams;
- complete UML Class Diagram;
- GoF participant mapping and pattern justifications;
- two UML Sequence Diagrams;
- one UML State Diagram;
- important design decisions and revision history; and
- a short team contribution statement.

15 Final Advice

Begin with the problem, not the pattern class names. Decide what work exists, who or what performs it, what changes during execution, what decisions route the work, which responsibilities should vary independently, and what must remain generic.

Then decide how the design patterns covered in COS 214 help solve those problems.

The strongest submissions will be the ones in which the research, scenario, Activity Diagrams, class design, pattern choices and runtime diagrams tell the same coherent story.